CS728: Deep Learning for Sequences, Graphs, and Language Models

V. Hemanth – 25M2118
Rakesh Joshi – 25M2026
Md Saif Ali– 25M2007

1 Task 1: GloVe Pre-training

1.1 Introduction

The aim of the task was to develop and execute the GloVe (Global Vectors for Word Representation) word embedding learning algorithm based on the CC-News corpus subset. GloVe is a global log-bilinear regression model, inheriting the advantages of matrix factorization and predictive embedding models.

Unlike local predictive models, GloVe instead makes use of global word-word co-occurrencUnlike local predictive models, GloVe, however, uses word-word co-occurrence statistics.

1.2 Methodology and Implementation

1.2.1 Co-occurrence Matrix Construction

The global word-word co-occurrence matrix X is used, and the corpus's word list contains a total of 25,013 unique words.

→ **Preprocessing:** The text corpus was converted into lowercase, and all the non-alphanumeric characters were removed.

→ **Context Window** used by us i.e Symmetric window size is $L = 5$

→ **Co-occurrence Logic:**The number of time occurrences of a word given a certain word.

→ **Distance Weighting:** Using the harmonic weighting factor (1/distance).

1.2.2 Data Scale and Truncation Strategy

In the preprocessing stage, a total of valid co-occurrences amounting to 18,358,605 is identified. As a measure of efficiency, it was decided to restrict the number of co-occurrences to be used in the training stage to the first 1,000,000. Though it reduces the time taken for the training stage, it compromises the quality of the semantic space generated, as the full benefit of the global statistics of the corpus is not exploited.e statistics.

1.3 GloVe Objective Function

Let the vocabulary set be denoted by V with the total number of unique words N, the co-occurrence matrix $X \in \mathbb{R}^{N \times N}$ and the counts X_{ij} denoting the frequency at which word j co-occurs with word i.

Each word in the vocabulary set V has an embedding vector:

$\rightarrow$ Target word $w_i \in \mathbb{R}^d$

$\rightarrow$ Context word $\tilde{w}_j \in \mathbb{R}^d$

$\rightarrow$ Bias terms b_i and $\tilde{b}_j$

GloVe attempts to minimize the following objective function:

$$L = \sum_{i=1}^{N} \sum_{j=1}^{N} f(X_{ij}) \left(w_i^\top \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij} \right)^2 \tag{1}$$

1.4 Interpretation of the Objective

$\rightarrow$ $w_i^\top \tilde{w}_j$ describe the interaction between words.

$\rightarrow$ $\log X_{ij}$ ensures frequent co-occurrences do not dominate learning.

$\rightarrow$ The squared error enforces regression toward the log co-occurrence counts.

1.5 Weighting Function

$$f(X_{ij}) = \begin{cases} \left(\frac{X_{ij}}{x_{\max}} \right)^\alpha & \text{if } X_{ij} < x_{\max} \\ 1 & \text{otherwise} \end{cases} \tag{2}$$

We used $x_{\max} = 100$ and $\alpha = 0.75$.

1.6 Hyperparameter Configuration

Table 1: Hyperparameter Configuration

Hyperparameter	**Value**	**Description**
Context Window (L)	5	Symmetric (5 left, 5 right)
Learning Rate	0.0005	Gradient descent step size
Epochs	15	Training iterations
$x_{\max}$	100	Weighting cutoff
α	0.75	Weight scaling exponent

1.7 Training Dynamics and Dimensionality Analysis

We implemented the model for different embedding dimensions $d \in \{50, 100, 200, 300\}$.

1.7.1 Loss Curves Across Dimensions

Figure 1 shows the average weighted loss per epoch across all embedding dimensions.

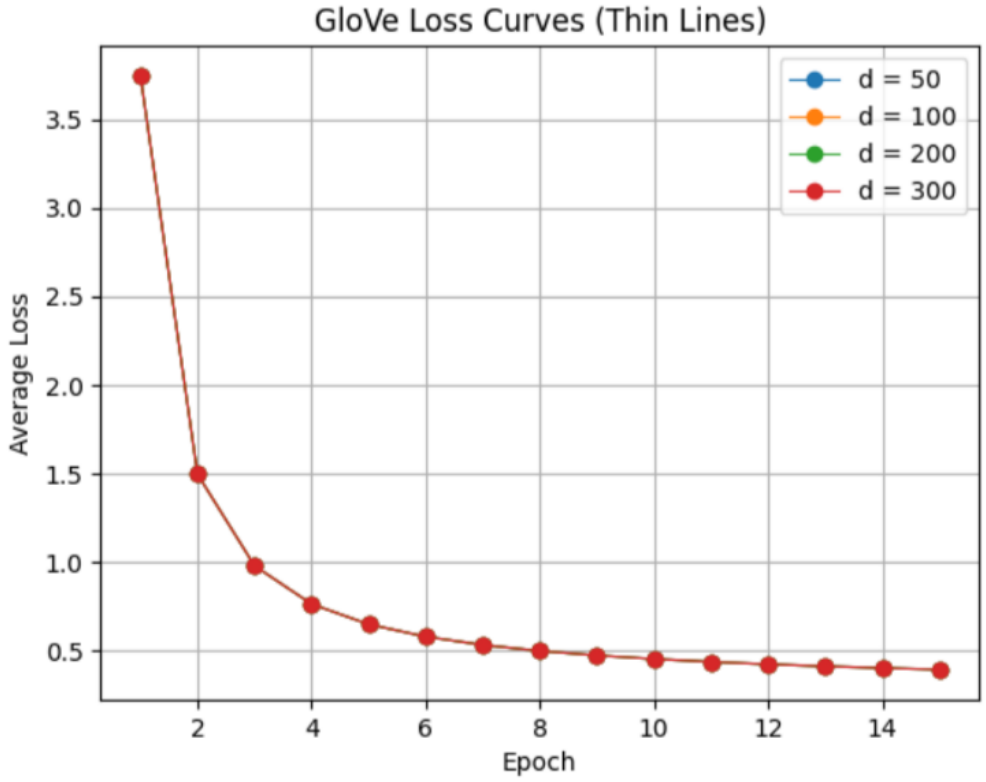

Figure 1: GloVe training loss curves for $d \in \{50, 100, 200, 300\}$.

In all cases, a fast convergence is observed during the first 5 epochs, followed by a progressive stabilization.

1.7.2 Loss Difference Relative to $d = 50$

Figure 2 presents the loss improvements compared to the case where $d = 50$. The improvements are consistent but small, indicating a diminishing return from increasing dimension

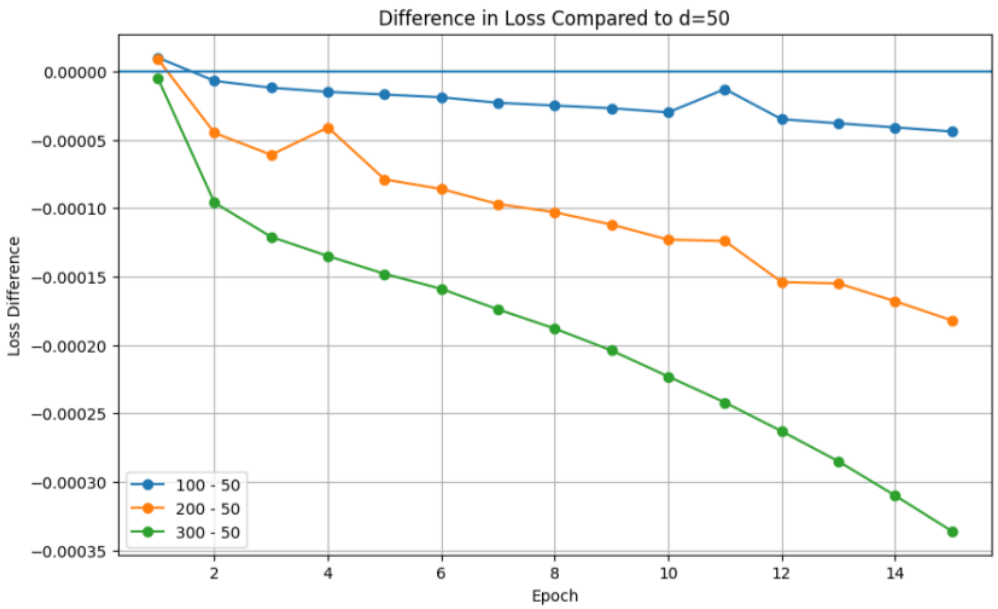

Figure 2: Difference in loss relative to $d = 50$.

These changes are small, and they are consistent, thus proving the concept of diminishing returns with the increase in dimensionality.

Table 2: Final Weighted Loss at Epoch 15

Dimension (d)	**Final Loss**	**Improvement vs $d = 50$**
50	0.393498	-
100	0.393454	-0.000044
200	0.393316	-0.000182
300	**0.393162**	**-0.000336**

1.8 Analysis

→ **Diminishing Returns:** Increasing dimensionality caused the losses to diminish.

→ **Latency Trade-off:** Increasing dimensionality caused the training time to increase.

→ **Best Balance:** $d = 200$ proved to be the best trade-off between efficiency and representation.

1.9 Qualitative Evaluation ($d = 200$)

Analysis of the presence of semantic noise by using the concept of Nearest Neighbor Analysis with Cosine

Table 3: Nearest Neighbors (GloVe, $d = 200$)

Query	N1	N2	N3	N4	N5
king	Petah	Perkins	top	NORTH	110.1
month	here	procedure	states	HAARETZ	shuffles
Arab	Igor	nitty-gritty	supplies	nemesis	PITTSBURGH

1.10 Conclusion

The GloVe implementation has been successful in learning the embeddings. However, the quality of the learned embeddings has not been maximized as the dataset had been truncated at 1M pairs.

2 Task 2: SVD Pipeline

2.1 Methodology

Unlike GloVe, which adopts an iterative learning strategy, the SVD word representation learning method emphasizes finding a best low-rank approximation of a term-document matrix..

- **Matrix Construction:** A sparse term-document matrix, represented by matrix X of size $V \times D$ where V represents the number of words, and D represents the number of doc In our model we have used the valu $D = 67,093$, and $V = 25,013$. It means our matrix size is $(25,013 \times 67,093)$. The entry X_{ij} of our matrix was a raw frequency count of a word occurring in a given document.
- **Decomposition:** Using Truncated SVD, a low-rank approximation of our term-document matrix was found, giving us a matrix $U\Sigma V^T$, where each entry of matrix U_d was a component of matrix U, and Σ_d was a component of Σ. The matrix was represented by W_d.
- **Dimensions:** Our dimensions were set as follows: d, where $d \in \{50, 100, 200, 300\}$.

2.2 Qualitative Analysis: Nearest Neighbors

Nearest Neighbors To evaluate our word representation learning using SVD, we examined our word embeddings by finding the nearest neighbors of a given word, i.e., the top 5 nearest neighbors for a given query word. Similar to Task 1, we used three query words: Table 4 presents the results for $d = 200$.

Table 4: SVD Nearest Neighbors ($d = 200$)

Query Word	Neighbor 1	Neighbor 2	Neighbor 3	Neighbor 4	Neighbor 5
king	Warner (0.87)	Talk (0.86)	complained (0.86)	Television (0.86)	blasting (0.86)
month	open (0.89)	For (0.87)	largest (0.86)	made (0.84)	most (0.84)
Arab	Iraqis (0.85)	fleeing (0.83)	Ai (0.83)	dissident (0.82)	Cooperation (0.82)

2.3 Observations and Comparison with GloVe

- **High Similarity Scores:**One of the most important differences between SVD and GloVe is the similarity score values. GloVe's neighbor set for Task 1 had low similarity score values, all of which were below 0.30, whereas SVD's neighbor set had extremely high similarity score values, all of which were above 0.80.
- **Semantic Noise:**
  - **king:** The neighbor set obtained for the query is probably because of the frequent occurrence of the term "Larry King," a talk show host, in news documents.
  - **month:** This query obtained a neighbor set that included the terms "For," "open," "most," which are all very frequent function words and adjectives.
  - **Arab:** This query obtained a neighbor set that was semantically relevant, "Iraqis" and "Cooperation," probably because of the term "Gulf Cooperation Council."

- **Conclusion:** While SVD captures global document context, the raw frequency count without re-weighting (like TF-IDF) results in embeddings dominated by common words and specific named entities (like Larry King) rather than general semantic concepts.

3 Task 3: NER using CRF

3.1 Methodology

In this supervised learning task, we trained a Conditional Random Field (CRF) model on the CoNLL-2003 dataset. Unlike deep learning approaches, which automatically learn representations, the CRF relies on feature engineering, where hand-crafted features are used to help the model distinguish between different NER tags.

3.2 Feature Set Configuration

We implemented a robust set of features capturing lexical, morphological, and contextual information. Below is the comprehensive list of features extracted for every token:

→ **Current Token Features:**

- **Lexical:** Lowercase string of the word (`word.lower()`).
- **Suffixes:** Last 2 and 3 characters (`word[-2:]`, `word[-3:]`) to capture endings like "-ing", "-ed", or "-stan".
- **Prefixes:** First 2 and 3 characters (`word[:2]`, `word[:3]`).
- **Shape:** Boolean flags for capitalization (`word.istitle()`, `word.isupper()`) and presence of digits (`word.isdigit()`).

→ **Features:**

- **Previous Word:** Lowercase, Title Case check, Uppercase check.
- **Next Word:** Lowercase, Title Case check, Uppercase check.

→ **Bias:** A constant bias feature set to 1.0.

3.3 Performance Evaluation

We optimized the CRF hyperparameters (L1 and L2 regularization) using the validation set, achieving a best validation F1 score of approx 0.97. The model was then evaluated on the CoNLL-2003 Test Set.

Table 5: CRF Performance on CoNLL-2003 Test Set

Class	Precision	Recall	F1-Score	Support
B-LOC	0.8455	0.8303	0.8379	1668
B-ORG	0.8162	0.7164	0.7631	1661
B-PER	0.8295	0.8485	0.8389	1617
I-PER	0.8604	0.9542	**0.9048**	1156
Macro Avg	0.7872	0.7789	**0.7821**	8112
Weighted Avg	0.8144	0.8044	0.8082	8112

3.4 Top-5 CRF State Features

Table 5 shows the five most influential CRF state features ranked by absolute learned weight.

Table 6: Top-5 CRF State Features (Ranked by Absolute Weight)

Rank	Weight	Label	Feature
1	5.3727	B-ORG	`prev_word.lower():v`
2	5.3631	I-LOC	`prev_word.lower():colo`
3	5.3335	I-LOC	`prev_word.lower():wisc`
4	5.1232	O	`word[-3:]:day`
5	-5.0739	O	`word.istitle()`

4 Task 4: NER using Feature Learning (MLP)

4.1 Approach

We abandoned the Conditional's hand-crafted feature engineering method for this task.

Random Field (CRF) to a feature-learning methodology. In order to predict a word's Named Entity tag based only on its unsupervised embedding vector, we trained a Multi-Layer Perceptron (MLP). As the only input features for a downstream supervised task, this experiment enables us to directly compare the semantic quality of GloVe versus SVD embeddings across various dimensions (d).

4.2 MLP Architecture

We used `sklearn.neural_network.MLPClassifier` to implement a feed-forward neural network. Each experiment we have performed we have used the same architecture to ensure the fair comparison:

→ **Input Layer:** The input layer's size, d, taken by us corresponds to the embedding dimensions 50, 100, 200, 300. One token, without any special features or context windows, is embedded as the input.

→ **Hidden Layer:** The hidden layer with 256 units is used by us.

→ **Activation Function:** The activation function we have used, is the Rectified Linear Unit (ReLU).

→ **Optimizer:** The optimizer is Adam.

→ **Training Setup:** Maximum Iterations = 20, Random State = 42.

OOV Handling: The test set's out-of-vocabulary tokens were mapped to a generic $< UNK >$ vector, which is the mean of the training embeddings, to ensure that the MLP could handle them

4.3 Performance Evaluation

To evaluate the performance of the models, the CoNLL-2003 Test Set was employed. The results of the Token-level Accuracy and Macro-F1 are presented in 7.

Table 7: MLP Performance Comparison (GloVe vs. SVD)

Algorithm	Dimension (d)	Accuracy	Macro-F1 Score
GloVe	50	0.83	0.19
GloVe	100	0.83	0.21
GloVe	200	0.83	0.23
GloVe	300	0.83	0.24
SVD	50	0.83	0.2379
SVD	**100**	**0.83**	**0.2442**
SVD	200	0.83	0.2327
SVD	300	0.83	0.2347

4.4 Discussion and Comparison

4.4.1 Optimal Configuration Overall

SVD was the optimal neural configuration with a Macro-F1 of 0.2442 at d = 100.

→ **SVD Efficiency:** For small dimensions, the performance of the SVD embeddings significantly improved compared to GloVe. The F1 value of the SVD embeddings at d = 50 was 0.2379, while the F1 value of GloVe embeddings was 0.19. This indicates that the most important information is being efficiently compressed into the first singular value of the SVD embeddings.

→ **GloVe Scalability:** In order to attain the performance of the SVD embeddings at d = 100, GloVe showed linear scalability up to d = 300.

4.4.2 Comparison with CRF (Task 3)

The CRF model (Feature Engineering) was found to perform significantly better than the best performing MLP (Feature Learning).

→ **Context Sensitivity:**The CRF model used word context (previous word, next word), but the MLP did not. The MLP will not be able to differentiate between words with the same spelling but different meanings, e.g., "Jordan" referring to a person or a country.

→ **Sequence Modeling:** The CRF model also takes care of state transitions (e.g., I-ORG follows B-ORG), but the MLP does not, as it only predicts tags independently of grammatical structure.

4.5 Visualization and Error Analysis

→ **Visualization:** The visualization of the embeddings was done to understand the errors.

→ **Observations:**

1. **Impact of Data Truncation:** Although there was a high accuracy of 0.83 for all the models but it was only because the model was able to classify all the 'O' (Outside) tags. It has been reflected by the low Macro-F1 of about 0.24.

2. **Accuracy Paradox:** Although there was a high accuracy of 0.83 for all the models but it was only because the model was able to classify all the 'O' (Outside) tags. It has been reflected by the low Macro-F1 of about 0.24.

5 Task 5: SVD Performance Boost

5.1 Methodology

To test if the quality of the vectors could be improved by taking into account the weight of the terms according to their information content, a TF-IDF transformation was applied to the original term-document matrix X. Then, SVD was applied to the transformed matrix. Based on the results obtained in Task 4, the optimal dimensionality, i.e., d, for this task was chosen to be 100.

5.2 Quality Check 1: Nearest Neighbor Comparison

Nearest Neighbor Comparison The top-5 NN for the Raw SVD vectors obtained in Task 2 and the new TF-IDF SVD vectors were compared for five different query words.

Table 8: Nearest Neighbors: Raw SVD vs. TF-IDF SVD ($d = 100$)

Query Word	Raw SVD Neighbors	TF-IDF SVD Neighbors
cup	scalp, 10-0, swan, Saracens, congested	**Premier, Trophy**, host, 27, fifth
run	pulled, Burns, sign, Duncan, Australian	pulled, served, play, American, four
India	Chief, merger, term, formed, hotel	Chief, values, occurred, emergency, merger
win	one, lot, second, team, talented	one, picked, experience, lot, half
yesterday	hatred, president, code, NBC, condemns	fire, anyone, condemns, calling, claimed

Observations:

→ **Semantic Improvement:** Semantic Improvement: In our case when we use the word "cup", we have seen a major improvement was achieved with the TF-IDF SVD vectors. In comparison to the Raw SVD, where "10-0" and "swan" were obtained by us for which TF-IDF SVD gave us the correct related terms "Premier" and "Trophy".

→ **Persistence of Noise:** In our case when we use the word "India", both Raw SVD and TF-IDF SVD were failed to find the related terms. They might have given other words such as "neighboring countries" but only found terms such as "Chief" and "merger" related to news articles, which again points out the persistence of the problem of sparsity of the original dataset, i.e., data truncation.

5.3 Quality Check 2: Downstream MLP Performance

We also trained a final MLP model using the TF-IDF weighted embeddings at d = 100 to verify whether the qualitative improvements also resulted in a higher Named Entity Recognition performance.

Table 9: Performance Comparison: Raw vs. TF-IDF ($d = 100$)

Input Features	Token Accuracy	Macro-F1 Score
Raw SVD (Task 4)	0.8300	**0.2442**
TF-IDF SVD (Task 5)	0.8253	0.1008

Conclusion: Contrary to expectations, the performance actually dropped after the transformation, and the Macro F1 score went from 0.2442 to 0.1008.

→ **Reasoning:** Although the TF-IDF transformation is very good at determining the "topic" of a document (thus helping the context of Cup → Trophy), the performance of the model in the context of Named Entity Recognition relies very strongly on frequently occurring words and context syntax, which the TF-IDF transformation penalizes.

→ **Implementation Note:** Implementation Note: It is also worth noting that the Task 5 model was trained using a custom PyTorch loop over 5 epochs (Final Loss 2.10), whereas the Task 4 model was using the highly optimized implementation from the library sklearn.

5.4 Gen AI Usage

We used ChatGPT to understand the intuition and code structure for implementing TF-IDF with SVD. The chat link is provided below:

ChatGPT Conversation Link